Git Worktrees

0 contexte perdu · N sujets en parallèle

Travailler intelligemment sur plusieurs branches simultanément

Équipe Abonnements · Mars 2026

Agenda

Partie 1 : Le problème

1. Le quotidien d'un dev multi-tâches
2. Les limites de `stash` et `checkout`

Partie 2 : La solution

3. Git Worktrees : le concept
4. Architecture et fonctionnement
5. Comparaison objective

Partie 3 : En pratique

6. Use cases concrets
7. `worktree-manager.sh`
8. Démo live

Partie 4 : Intégration IA

9. Claude Code · OpenCode · Cursor
10. Installation en 2 minutes
11. Adoption équipe

Partie 1

Le problème

Le quotidien d'un dev multi-tâches

Tu travailles sur **ALM-1234** depuis 3 heures. Code en cours, non commité.

Scénario 1 : Le Slack qui tue

> "@dev il y a une régression en prod sur le paiement, tu peux checker ?"

Scénario 2 : La PR qui traîne

> "Peux-tu review ma PR ALM-5678 ? Je suis bloqué en attendant le merge."

Scénario 3 : Le backlog qui s'accumule

> Ticket de hotfix urgent + feature en cours + review à faire = 3 branches

La réaction naturelle... et ses problèmes

Option A : `git stash`

```
git stash
git checkout fix/urgent-hotfix
# ... travail ...
git checkout feat/ALM-1234
git stash pop
# 💥 conflits ? perte de contexte ?
```

Problèmes :

- Contexte mental perdu
- Stash stack = enfer à gérer
- Risque de conflit au pop
- Fichiers ouverts dans l'IDE = chaos

Option B : `git checkout`

```
# Impossible si des fichiers
# non commités sont présents
git checkout feat/other
# erreur: vos modifications locales
# seraient écrasées
```

Problèmes :

- Bloqué si changements en cours
- Oblige un commit prématuré
- Commit en mode "wip" = historique pollué
- Dépendances à réinstaller ?

Partie 2

La solution

Git Worktrees : le concept

Un **worktree** est un répertoire de travail supplémentaire lié au même dépôt Git.

> Un seul `.git` → plusieurs dossiers de travail indépendants, chacun sur sa propre branche.

```
monrepo/
├── .git/                          ← objet store partagé (commits, blobs, refs)
├── src/                           ← branche main (worktree principal)
│
└── .worktrees/
    ├── feat/ALM-1234/ma-feature   ← branche feat/ALM-1234  ← worktree isolé
    └── fix/ALM-5678/hotfix        ← branche fix/ALM-5678   ← worktree isolé
```

Chaque worktree a son propre répertoire de travail, son propre index Git, ses propres fichiers non commités, **complètement indépendants**.

Comment ça marche ?

Le `.git` partagé

Tous les worktrees partagent :

- L'historique des commits
- Les objets Git (blobs, trees)
- Le stash (accessible partout)
- Les remotes

Chaque worktree a sa propre :

- **Branche active**
- **Index (zone de staging)**
- **Working directory**
- **HEAD**

Contrainte fondamentale

> Une même branche ne peut être checkoutée que dans **un seul worktree** à la fois.

```
# Créer un worktree (git natif)
git worktree add .worktrees/feat/ALM-1234 \
  -b feat/ALM-1234

# Lister
git worktree list

# Supprimer
git worktree remove .worktrees/feat/ALM-1234
git worktree prune
```

Pas besoin de re-cloner le repo

C'est l'avantage clé souvent sous-estimé.

Sans worktree

Travailler sur une branche isolée = cloner le repo dans un autre dossier :

```
git clone git@github.com:org/repo.git \
  ~/workspace/repo-hotfix
cd ~/workspace/repo-hotfix
composer install  # 3-5 minutes
cp .env.example .env
# ... setup docker, config ...
```

Résultat : doublon complet du repo, dépendances re-téléchargées, config à refaire.

Avec worktree

```
bash worktree-manager.sh create fix/hotfix
# vendor/ → symlink (0 copie)
# .env*   → copié automatiquement
# Prêt en quelques secondes
```

Un seul `.git`, un seul `vendor/`, zéro duplication. Les worktrees partagent les objets Git et les dépendances.

Utilisé en production : le Core Team de Symfony travaille explicitement avec des worktrees pour contribuer au framework et maintenir plusieurs versions majeures (6.x, 7.x, 8.x) en parallèle sur le même clone.

Comparaison objective

Critère	`git stash`	`git checkout`	Git Worktree
Contexte de travail préservé	Partiel	Non	**Oui, totalement**
Plusieurs branches simultanées	Non	Non	**Oui**
IDE reste ouvert sur les deux	Non	Non	**Oui**
Commit en mode "wip"	Non	Souvent	**Non**
Dépendances (`vendor`)	Partagées	Partagées	**Symlink auto**
Fichiers `.env`	Manuel	Manuel	**Copie auto**
Clone supplémentaire nécessaire	Non	Non	**Non**
Rollback risqué	Oui	Non	**Non**

Partie 3

En pratique

Use case 1 : Multi-sujets en parallèle

Situation : tu développes en parallèle ALM-1234 (feature) et ALM-5678 (chore), ou bien tu es en train de travailler sur ALM-1234 et une idée d'amélioration te vient pour ALM-5678 que tu veux implémenter.

```
# Créer les deux worktrees
bash worktree-manager.sh create feat/ALM-1234/ma-feature
bash worktree-manager.sh create chore/ALM-5678/refacto

# Résultat
.worktrees/
├── feat/ALM-1234/ma-feature   ← ton IDE ouvert ici
└── chore/ALM-5678/refacto     ← autre fenêtre IDE

# Aller d'un sujet à l'autre : juste changer de fenêtre de terminal ou d'IDE
# Pas de stash, pas de checkout, pas de perte de contexte
```

> Chaque worktree est un dossier indépendant, ouvre-les dans **deux fenêtres IDE ou deux terminaux**.

Use case 2 : Feature longue + feature rapide

Situation : tu es en plein développement d'une feature qui prend du temps (ALM-1234, refacto profonde). Une nouvelle feature simple et rapide arrive (ALM-9000, 1h de dev).

Problème sans worktree

- Stash ou commit "wip" pour switcher
- Risque de mélanger les changements
- Perte de contexte sur la feature longue
- La feature rapide peut trainer car pénible à isoler

Avec worktree

```
# Feature longue → worktree dédié
bash worktree-manager.sh create \
  feat/ALM-1234/gros-refacto

# Feature rapide → autre worktree
bash worktree-manager.sh create \
  feat/ALM-9000/ajout-rapide

# Dev la feature rapide → PR → merge
# Reviens sur la feature longue
# → exactement où tu l'avais laissée
```

> Chaque feature avance à son rythme, dans son contexte. La feature rapide peut être livrée sans attendre la longue.

Use case 3 : Hotfix d'urgence

Situation : 3h de dev sur ALM-1234, non commité. Régression prod signalée.

Sans worktree

```
# Option risquée
git stash  # perdre le contexte
git checkout -b fix/urgent
# ... fix ...
git push
git checkout feat/ALM-1234
git stash pop  # 💥 possible conflits
```

Temps perdu à retrouver le contexte.

Avec worktree

```
# En 10 secondes
bash worktree-manager.sh create fix/urgent

# Nouvelle fenêtre terminal
cd .worktrees/fix/urgent
# ... fix ...
git push

# Retour : ta fenêtre feat/ALM-1234
# est EXACTEMENT comme tu l'as laissée
```

Contexte ALM-1234 : intact. Zéro interruption.

Use case 4 : Code Review sans interruption

Situation : Quelqu'un te demande de reviewer la PR de ALM-9999.

```
# Checkout de la PR dans un worktree dédié
bash worktree-manager.sh create feat/ALM-9999/feature-a-reviewer

cd .worktrees/feat/ALM-9999/feature-a-reviewer
# Lire le code, lancer les tests, tester manuellement
# vendor/ déjà symlinké → rien à réinstaller
```

> Tu reviews dans une fenêtre, tu continues de développer dans une autre.
> Aucun fichier de ton dev en cours ne bouge.

Use case 5 : Migrate (j'ai déjà commencé à coder)

Situation : Tu as codé directement sur `main` sans créer de branche. Oups.

```
# La commande migrate = stash → create → stash pop automatique
bash worktree-manager.sh migrate feat/ALM-1234/ma-feature

# Ce qui se passe en coulisse :
# 1. git stash push --include-untracked  (sur main)
# 2. git worktree add -b feat/ALM-1234 .worktrees/feat/...
# 3. git stash pop  (dans le nouveau worktree)
# 4. vendor symlink + .env copy
```

> Ton code "accidentellement sur main" se retrouve proprement dans un worktree isolé sur sa propre branche, **en une commande**.

worktree-manager.sh : C'est quoi ?

Un **script Bash standalone** qui encapsule `git worktree` avec les automatismes du projet, utilisable directement en ligne de commande ou via un assistant IA.

```
$ bash worktree-manager.sh help

Git Worktree Manager

Usage: worktree-manager.sh <command> [options]

Commands:
  create <branch> [from]   Create new worktree (copies .env files automatically)
                           (from defaults to main)
  migrate <branch> [from]  Move uncommitted changes to a new worktree
                           (stash → create worktree → stash pop)
  list | ls                List all worktrees and current status
  switch | go [name]       Switch to a worktree (or 'main' to go back)
  copy-env | env [name]    Copy .env files from main repo to worktree
  cleanup | clean          Interactively remove inactive worktrees
  help                     Show this help

Notes:
  - Worktrees are stored in .worktrees/ (auto-added to .gitignore)
  - .env files are copied automatically on create
  - Each worktree has its own isolated working directory
  - You can push/commit from any worktree independently
```

worktree-manager.sh : Commandes en détail

`create <destination> [source]`

```
# Branche depuis main (défaut)
bash worktree-manager.sh create feat/ALM-1234/desc
#  └─ destination : feat/ALM-1234/desc (nouvelle branche)
#  └─ source      : main (défaut)

# Branche depuis une autre branche
bash worktree-manager.sh create fix/bug develop
#  └─ destination : fix/bug (nouvelle branche)
#  └─ source      : develop (base de départ)
```

`copy-env [branche]`

Copie les `.env*` depuis la racine du repo **vers un seul worktree** spécifié. Si aucun nom donné, auto-détecte le worktree courant.

```
# Vers un worktree précis
bash worktree-manager.sh copy-env feat/ALM-1234/desc
# Depuis l'intérieur du worktree
bash worktree-manager.sh copy-env
```

`cleanup`

Liste **tous les worktrees inactifs** (hors worktree courant) et demande confirmation avant de les supprimer. Lance ensuite `git worktree prune`.

```
$ bash worktree-manager.sh cleanup

Checking for worktrees to remove...

• feat/ALM-1234/desc  (branch: feat/ALM-1234)
• fix/old-hotfix      (branch: fix/old-hotfix)

Remove 2 worktree(s)? (y/n)
```

> Ne supprime **pas** le worktree dans lequel tu te trouves. Demande toujours confirmation.

Démo live

```
bash worktree-manager.sh create feat/ALM-XXXX/demo
```

> Création · Isolation · Symlink vendor · Copy .env

```
bash worktree-manager.sh list
```

```
bash worktree-manager.sh cleanup
```

Partie 4

Intégration IA

L'idée : l'IA gère les worktrees pour toi

Au lieu de taper les commandes manuellement, tu **demandes à ton assistant IA** de créer, lister, ou nettoyer les worktrees.

Claude Code CLI

Slash command `/git-worktree`
→ `~/.claude/commands/`
→ `SKILL.md`

OpenCode

Plugin TypeScript
→ `~/.config/opencode/`
→ outil `worktree`

Cursor

Rules `.mdc`
→ `.cursor/rules/`
→ contexte automatique

Le **script** `worktree-manager.sh` est le moteur commun. Chaque outil IA l'expose différemment.

Claude Code CLI : `/git-worktree`

```
# Dans ~/.claude/commands/git-worktree.md

Quand l'utilisateur demande de gérer les worktrees,
exécute le script via Bash :

bash ~/.local/share/git-worktree/worktree-manager.sh <command> [args]
```

Utilisation :

```
/git-worktree create feat/ALM-1234/ma-feature
/git-worktree list
/git-worktree migrate feat/ALM-1234/ma-feature
/git-worktree cleanup
```

> Claude lit le SKILL.md, comprend les commandes disponibles, et exécute le bon bash automatiquement.

OpenCode : Plugin TypeScript

L'install enregistre le plugin **et met à jour** `config.json` **automatiquement**.

```
// plugin installé dans :
// ~/.config/opencode/plugins/git-worktree/plugin.ts

// config.json mis à jour automatiquement :
{
  "plugins": [
    "~/.config/opencode/plugins/git-worktree/plugin.ts"
  ]
}
```

Important

OpenCode peut utiliser son bash natif au lieu du plugin si la demande est trop générique.

Toujours préférer la commande directe :

```
wt create feat/ALM-1234/ma-feature
```

OpenCode exécutera le script via son outil bash, avec le bon chemin, symlink vendor et copie .env.

Installation en 1 commande

```
curl -sSL https://raw.githubusercontent.com/nayzo/ai-skills/main/git-worktree-generic/install.sh | bash
```

Le script installe tout et pose une seule question :

```
Quel outil IA utilises-tu ? (plusieurs choix possibles, ex: 1 3)
  1) Claude Code
  2) Cursor
  3) OpenCode
  4) Aucun (standalone uniquement)

Ton choix :
```

Configure automatiquement selon la réponse : alias `wt`, intégration IA, détection du shell (zsh / bash / fish) et de l'OS (Mac / Linux).

> Le script et la commande `wt` sont installés **dans tous les cas**, quel que soit l'outil IA choisi. L'intégration IA est optionnelle et vient en plus.

Mise à jour :

```
wt update   # re-télécharge la dernière version depuis GitHub
```

Adoption équipe : Proposition

Ce que je propose

- **Install one-liner** à partager dans le README équipe :

```
curl -sSL https://raw.githubusercontent.com/nayzo/\
ai-skills/main/git-worktree-generic/install.sh | bash
```

- **Makefile target** (optionnel) :

```
worktree: ## Usage: make worktree BRANCH=feat/ALM-XXXX/desc
  @[ -n "$(BRANCH)" ] || \
    (echo "Usage: make worktree BRANCH=..."; exit 1)
  wt create $(BRANCH)
```

- **Skill IA** configuré au choix : Claude Code, Cursor, OpenCode

Bénéfices équipe

- Aucune dépendance externe, git natif, Git ≥ 2.5 (2015)
- Fonctionne avec Docker : les containers partagent le même vendor
- Chaque dev garde ses worktrees dans `.worktrees/` (gitignored), pas d'impact sur les autres
- Adoption progressive : le script peut être utilisé sans IA, standalone

Questions ?

Script disponible : `ai-skills/git-worktree-generic/`

Script standalone
`worktree-manager.sh`

Skill IA
Claude Code · OpenCode · Cursor

Makefile
`make worktree BRANCH=...`